



静的型付けの哲学

-Ruby vs. TypeScript-

作成者: 石田 隼人

最終更新日: 2025年06月24日

英語版は [こちら](#)

自己紹介

- 各種アカウント
 - GitHub: [@hayat01sh1da](#)
 - X: [@hayat01sh1da](#)
 - LinkedIn: [@hayat01sh1da](#)
 - Speaker Deck: [@hayat01sh1da](#)
 - Docswell: [@hayat01sh1da](#)
- 職業: ソフトウェアエンジニア
- 趣味
 - 言語学
 - カラオケ
 - 音楽鑑賞
 - 映画鑑賞
 - 猫飼育



免許 / 資格

- 英語
 - TOEIC® Listening & Reading 915点: 2019年12月 取得
- エンジニアリング
 - 情報セキュリティマネジメント: 2017年11月 取得
 - 応用情報技術者: 2017年06月 取得
 - 基本情報技術者: 2016年11月 取得
 - IT パスポート: 2016年04月 取得
- その他
 - 珠算2級: 2002年06月 取得
 - 暗算3級: 2001年02月 取得

T S

スキルスタック

- 自然言語
 - 日本語: 母語
 - 英語: ビジネスレベル
- 開発
 - **バックエンド: Ruby(Ruby on Rails), Python**
 - フロントエンド: TypeScript + React.js, TypeScript + Vue.js
 - **データベース: MySQL, PostgreSQL, MongoDB**
 - **アーキテクチャー: モノリス, モジュールモノリス**
 - ホスティング: AWS ESK
 - バージョン管理: Git, GitHub
 - **CI/CD: GitHub Actions, ArgoCD**
 - 監視: Datadog, Sentry

VS.

T S

職歴

期間	業種	職種	業務内容
2021年08月- Present	EdTech SaaS	ソフトウェア エンジニア	• バックエンド開発 • フロントエンド開発 • CI/CD 運用・保守 • 週次リリースマネージャー • トラブル対応 • ドキュメンテーション • 技術負債解消 • 技術ブログ記事執筆 • LT 登壇
2020年02月- 2020年12月	チャットボット系 SaaS	バックエンド エンジニア	• バックエンド開発 • ドキュメンテーション • 技術負債解消 • 代替 API 検証

職歴

期間	業種	職種	業務内容
2018年06月 - 2020年01月	受託開発	ソフトウェア エンジニア	• バックエンド開発 • フロントエンド開発 • トラブル対応 • QA • 技術ブログ記事執筆
2016年04月 - 2018年01月	SES	システムエンジニア	• 全社アカウント管理者 • Windows Server 運用・保守 • セキュリティ管理者助手 • 翻訳・通訳

国際交流活動

- 大学時代
 - 英語学ゼミ(マスメディア英語)
 - 国際交流サークル兼部(2年次)
 - [内閣府主催国際交流プログラム](#)(2013年 - 2016年)
 - 日本語学の授業のS単位取得(最終年次)
- 海外生活
 - オーストラリアでのワーキングホリデー(2014年04月 - 2015年03月)
 - 2ヶ月間のシドニーの語学学校通学
 - 6ヶ月間の [Hamilton Island Resort](#) での就労
 - 1ヶ月間の NSW 州の [St Ives High School](#) での日本語教師アシスタントボランティア活動
- その他活動
 - 英語での日々の日記(2014年04月 - 現在)
 - [Sunrise Toastmasters Club](#) 参加(2017年02月 - 2018年03月)
 - [Vital Japan](#) 参加(2018年01月 - 2019年07月、2022年10月 - 2023年02月)
 - 英語自己学習
 - Ruby 関連カンファレンス参加

VS.

T S

コンテンツ

1. [静的型付けの基本おさらい](#)
2. [型付けエコシステム](#)
3. [型付けの哲学](#)
4. [まとめ](#)
5. [参考文献](#)

VS.

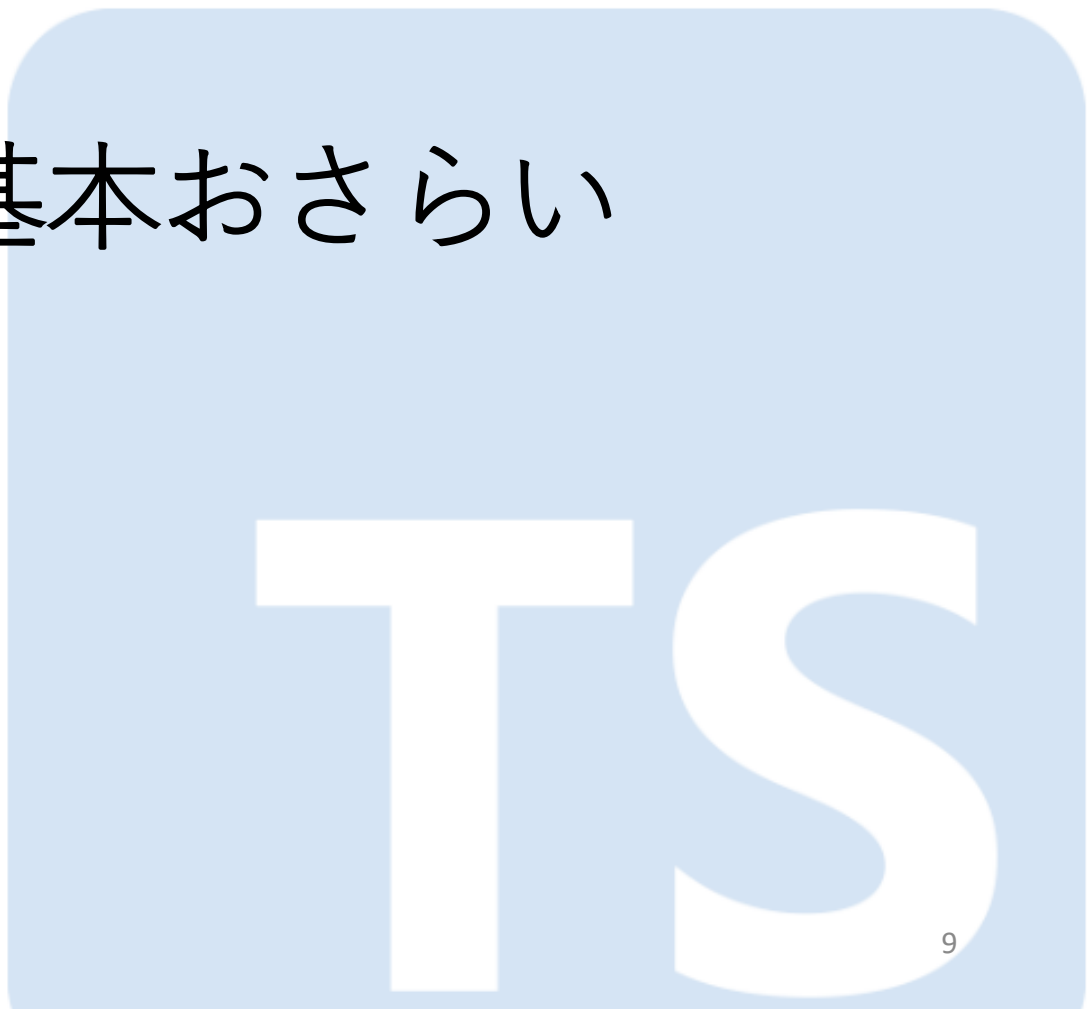


TS



1. 静的型付けの基本おさらい

vs.



TS

1. 静的型付けの基本おさらい

トレードオフ

- 👍 **未定義動作**によってもたらされるシステムやプログラムの**異常終了**や**セキュリティホール**から保護する
- 👍 実行前にタイプエラーを検知する
- 👍 大規模なソースコードをバグ・型の不整合・危険なリファクタリングなどの「鼻から悪魔」からより効率的に保護する
- 🗨️ JavaScript のような処理機構では、例えば `x.foo()` を実行するたびにレシーバーの型を検査するので、**大きなオーバーヘッドによるパフォーマンス悪化**を招くケースがある
- 🗨️ 開発者は**ソースコードだけでなく型定義の管理**もしなければならない
- 🗨️ 開発者は**静的型付けのメカニズムを理解するための学習コスト**を払わなければならない

1. 静的型付けの基本おさらい

分類	型検査のタイミング	言語
静的型付け	コード実行前	• TypeScript • C • Rust • <u>Java</u> • <u>Swift</u> etc.
動的型付け	コード実行中	• Ruby • JavaScript • <u>Java</u> • <u>Swift</u> etc.



2. 型付けエコシステム

vs.



TS

2. 型付けエコシステム

Ruby

機構	Gem	説明
RBS	RBS (Ruby Signature)	Ruby プログラムの構造を記述するための公式 Gem。Ruby の DSL ではなく別の言語である
	Steep	RBS 上の型定義を元に型検査を行う
	TypeProf	Ruby コードを元に RBS 上の型定義の推論を行う
	rbs-inline	Ruby コード上のインラインコメントを元に RBS を生成する
RBI	RBI (Ruby Interface)	Ruby の I/F ファイルを Sorbet に提供するための Ruby API を公開する
	Sorbet	既存の Ruby コードに漸進的な型付けとエラーや提案の迅速な応答を提供する
	Tapioca	Ruby コードだけでなく、Gem・Rails・DSL などを読み込んで RBI ファイルをコンパイルする

2. 型付けエコシステム

Ruby: RBS + Steep

例) RBS + Steep

person.rb

```
1 require_relative './phone'
2 require_relative './email'
3
4 class Person
5   attr_reader :name, :contacts
6
7   def initialize(name:)
8     @name = name
9     @contacts = []
10  end
11
12 def guess_country()
13   contacts.map {
14     |it| it.country if it.is_a?(Phone)
15   }.compact.first
16 end
17 end
```

VS.

\$ rbs prototype rb person.rb

\$ steep check
Type checking files:
...
No type error detected. 🍵

sig/person.rbs

```
1 class Person
2   @name: untyped
3
4   @contacts: untyped
5
6   attr_reader name: untyped
7
8   attr_reader contacts: untyped
9
10  def initialize: (name: untyped) -> void
11
12  def guess_country: () -> untyped
13 end
```

2. 型付けエコシステム

Ruby: RBS + Steep

例) rbs-inline + Steep

person.rb

```
1 #rbs_inline::enabled
2 require_relative './phone'
3 require_relative './email'
4
5 class Person
6   attr_reader :name #: String
7   attr_reader :contacts #: Array[Phone | Email]
8
9   # @rbs::name: String
10  # @rbs::addresses: Array[String]
11  # @rbs::return: void
12  def initialize(name:)
13    @name = name
14    @contacts = []
15  end
16
17  # :() -> (String | nil)
18  def guess_country()
19    contacts.map {
20      |it| it.country if it.is_a?(Phone)
21    }.compact.first
22  end
23 end
```

\$ rbs-inline --output person.rb
Generated 1 RBS files under
sig/generated

\$ steep check
Type checking files:
...
No type error detected. 🍵

sig/generated/person.rbs

```
1 class Person
2   @name: String
3
4   @contacts: Array[Phone | Email]
5
6   attr_reader name: String
7
8   attr_reader contacts: Array[Phone | Email]
9
10  def initialize:(name: String) -> void
11
12  def guess_country:() -> (String | nil)
13 end
```

2. 型付けエコシステム

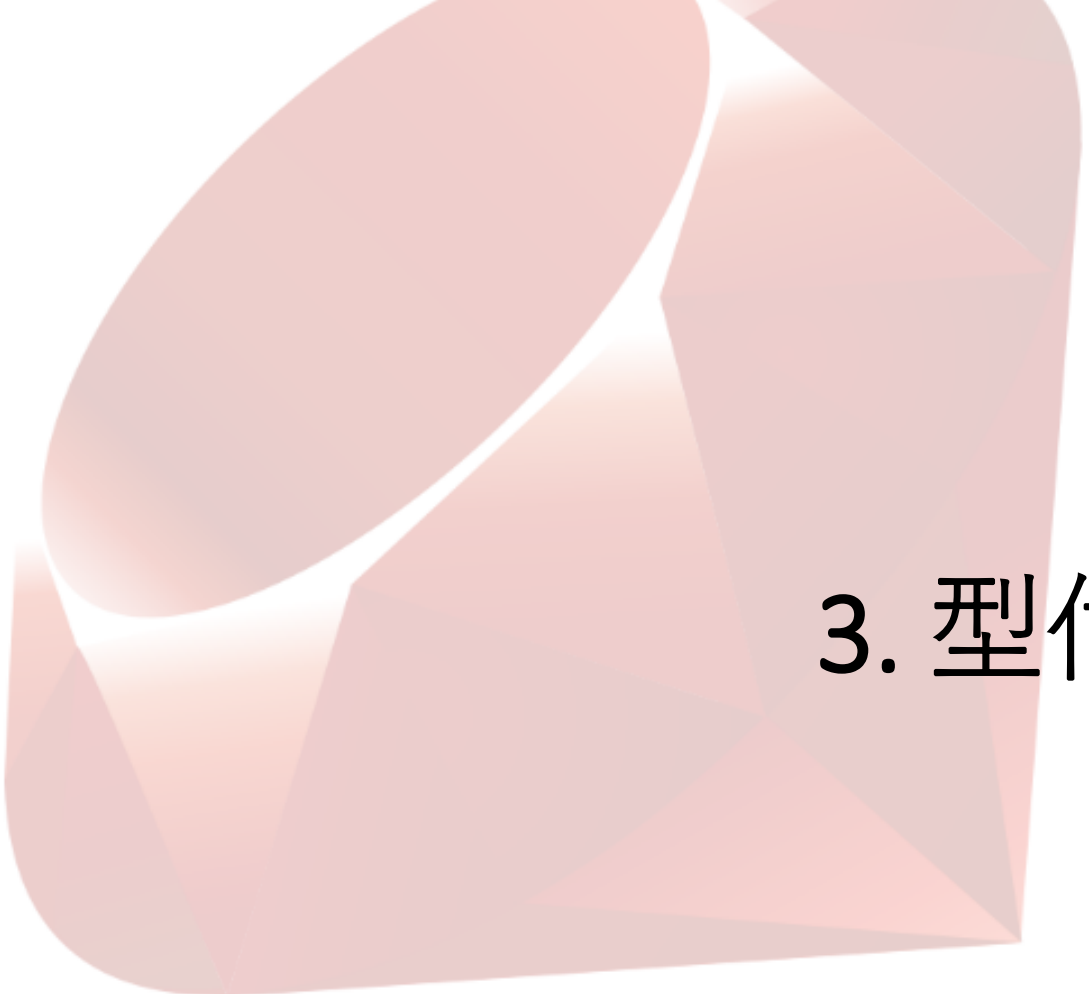
TypeScript

Ruby は構文の後方互換性を担保するために RBS 以外のエコシステムを取り入れていない。

一方、TypeScript は [Generics](#) ・ [Object Types](#) ・ [Interfaces](#) を大胆に取り入れて JavaScript のエコシステムを変化させた。

この大きな変更は「**既存コードに変更を加えずに**型検査を導入しなければいけない」という思い込みを覆したと、Ruby コミッターの松本宗太郎氏 ([@sotaro](#)) は [あるブログ記事](#) で言及した。

彼は JavaScript がすでに**豊富なライブラリや高機能な開発環境が整っていたため、既存のコードベースの TypeScript への移行が可能だった**とも述べた。



3. 型付けの哲学

vs.



TS

3. 型付けの哲学

Ruby

Heroku 社の Developer Advocate である [Jonan Scheffler 氏](#) が執筆した [あるブログ記事](#) で、Ruby の開発者であるまつもとゆきひろ氏(通称 Matz)が静的型付けについて以下のように言及している。

以下の会話は、氏が**豊かなコーディング体験とその後方互換性を保つことでもたらされる開発者の喜びを重視している**ことを証明する。

Jonan) それでは、JIT の文脈で Ruby 3 で導入されるいくつかの変更点と静的型付けについてすでにお話しされましたが、静的型付けについて教えてください。

Matz) ええ、Ruby 3 という3番目のメジャーバージョンの目標はダックタイピングを担保しつつ静的型付けを導入することなので、まあある種のソフトタイピングのような構造ですね。型付け機構の主目的は**早期のエラー検知**です。なので、この**静的型付けや型付けインターフェースの導入は既存のランタイムに影響はありません**。

3. 型付けの哲学

Ruby

事実、[RBS](#) は Ruby そのものからは独立したエコシステムであり、signature は Ruby とは異なる言語である。

`xxx.rbs` は対応する `xxx.rb` とは分離しており [rbs-inline](#) は型定義をコメントとして扱うので、既存の Ruby コードには影響を与えない(インライン型定義コメントがコメント以上の意味を持ってしまうが)。

私見では、[rbs-inline](#) のコンセプトは以下の点でとても開発者に優しいと考える。

- 静的型付け定義が仕様ドキュメントの役割を果たす
- 特にウェブ API として使われているアプリケーションでは、返却値の型は明示されて然るべきである
- [rbs-inline](#) のインラインコメントは文字通り一石二鳥である

3. 型付けの哲学

TypeScript

Ruby コミッターの遠藤侑介氏の著書『型システムのしくみ - TypeScript で実装しながら学ぶ型とプログラミング言語』の p.3 にて、「TypeScript は、無理がなく従来の JavaScript のコーディング体験を損なわない範囲でのみ望まない挙動の検知を行う」という TypeScript の哲学に言及している。

例えば、以下の後方定義変数はどちら型検査時にはエラーを発生しないが、後者実行時に **ReferenceError** が発生する。

a.ts

```
1 const f = () => x;
2 const x = 1;
3 console.log(f());
```

b.ts

```
1 const f = () => x;
2 console.log(f());
3 const x = 1;
```

WSL

```
Check file:// a.ts $ deno check a.ts
1 $ deno run -A a.ts
Check file:// b.ts $ deno check b.ts
error: Uncaught (in promise) ReferenceError: Cannot access 'x' before initialization
const f = () => x;
               ^
    at f (file:// b.ts:1:17)
    at file:// b.ts:2:13
```

3. 型付けの哲学

TypeScript

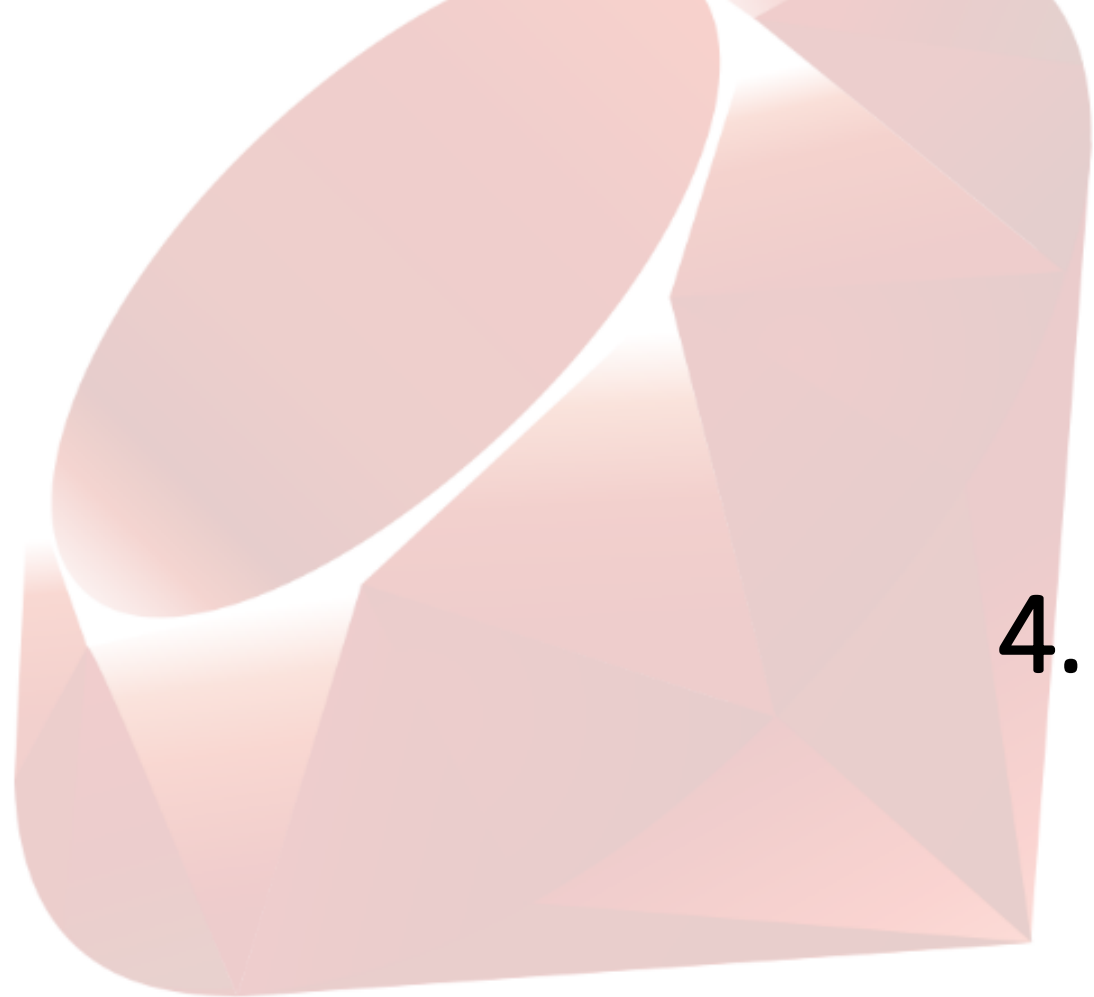
この現象は、**未初期化変数が定義済みの関数から参照出来てしまうこと**を表している。

このような型不安全は型検査機構で検知されるべきであるが、TypeScript は敢えて無視している。

それは何故か？

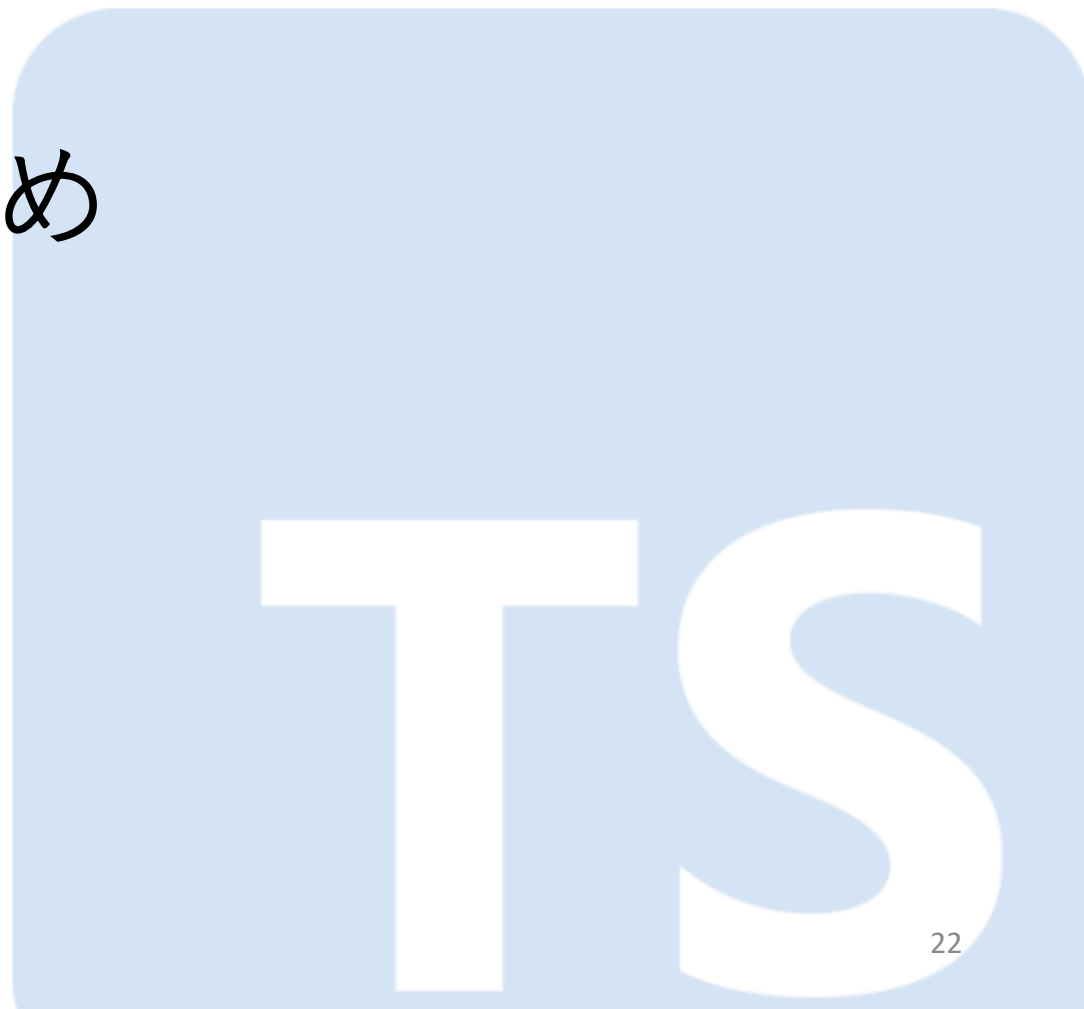
著者によると、JavaScript は従来この手の実装を許容してきたので、**TypeScript で未初期化変数に対する制約を入れると JavaScript との後方互換性を破壊してしまう可能性がある。**

Ruby がそうであるように、TypeScript もまた従来のコーディング体験の維持を重要視していることが分かる。



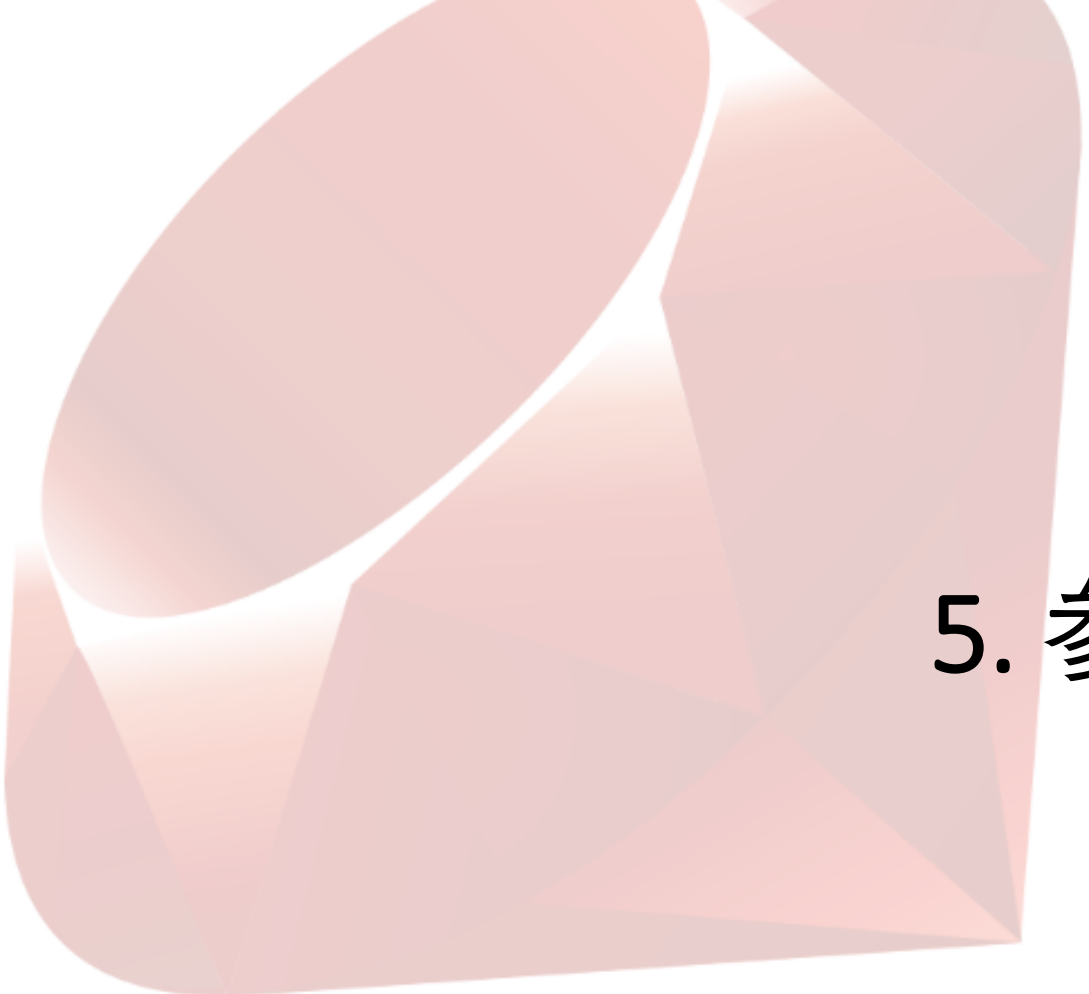
4. まとめ

vs.



4. まとめ

- Ruby と TypeScript はともに後方互換性を維持することによって従来のコーディング体験を保証することに粉骨砕身している
- TypeScript は [Generics](#) ・ [Object Types](#) ・ [Interfaces](#) の大胆な導入によって、静的型付け周りのエコシステムの統合に成功した
- Ruby は [RBS](#)(Ruby Signature) と [RBI](#)(Ruby Interface) という、静的型付けの2つの大きな勢力が存在する。まだ「自然淘汰」「適者生存」の決着はついていない
- いずれにしろ、開発者は認識している以上の恩恵を静的型付けから享受できるので、その導入はかなり価値の高いものである



5. 参考文献



TS

5. 参考文献

- [いかにして動的型付けの Ruby に静的な型検査を持ち込むか？ Steep と RBS が目指すもの](#)
 - 最終アクセス日: 2025年06月06日
- [RubyKaigi 2025 初参加エンジニアのための Ruby 静的型付けガイド](#)
 - 最終アクセス日: 2025年06月06日
- [The State of 静的型付け in Ruby in 2025](#)
 - 最終アクセス日: 2025年06月06日
- [Why 静的型付け is Essential for Efficient Ruby and Rails Development](#)
 - 最終アクセス日: 2025年06月06日
- [静的型付け in Ruby: It's Not a Cult, It's a Choice \(You Should Probably Make\)](#)
 - 最終アクセス日: 2025年06月06日

5. 参考文献

- [Ruby 3 × 3: Matz, Koichi, and Tenderlove on the future of Ruby Performance](#)
 - 最終アクセス日: 2025年06月06日
- [RBS\(Ruby Signature\)](#)
 - 最終アクセス日: 2025年06月06日
- [Steep](#)
 - 最終アクセス日: 2025年06月06日
- [TypeProf](#)
 - 最終アクセス日: 2025年06月06日
- [rbs-inline](#)
 - 最終アクセス日: 2025年06月06日
- [RBI\(Ruby Interface\)](#)
 - 最終アクセス日: 2025年06月06日

VS.

T S

5. 参考文献

- [Sorbit](#)
 - 最終アクセス日: 2025年06月06日
- [Tapioca](#)
 - 最終アクセス日: 2025年06月06日
- [TypeScript Official Documents](#)
 - 最終アクセス日: 2025年06月06日
- [10 Must-Know TypeScript Features to Improve Your Coding Efficiency](#)
 - 最終アクセス日: 2025年06月06日
- 遠藤侑介、『型システムのしくみ - TypeScript で実装しながら学ぶ型とプログラミング言語』、東京、ラムダノート株式会社、2025年

VS.



EOD

